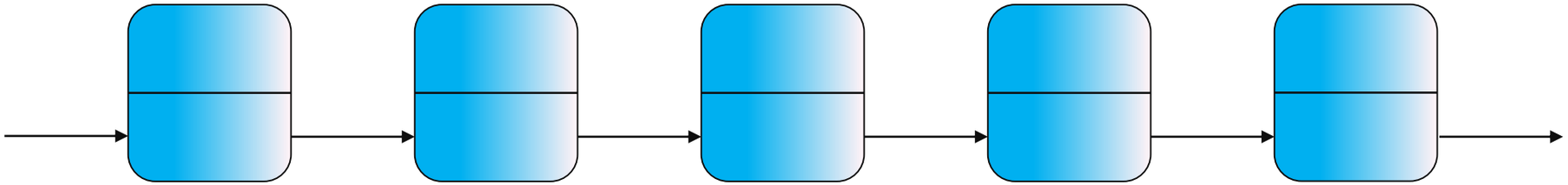


Basic Data Structures

Institute of
Algorithms and Theory

Yannic Maus

Algorithms:

Step-by-step instructions for solving (computational) problems.

Goal: Find the most efficient solution methods possible.

Data Structures:

Methods for storing and organizing data.

Goal: Enable efficient access and modification of data.

First central question of this course: *What does it mean to be efficient?*

Algorithms, Data Structures, and efficiency go hand-in-hand and cannot be separated.

"Chicken or egg, which came first?" *We first focus on algorithms and efficiency …*

Abstract Data Types (ADTs)

Abstract Data Structure/Abstract Data Type (ADT):

- Specification which kind of data should be stored
- Specification which operations should be executeable (e.g. insert(int x))
  - With the semantics of each operation

Data Structure:

- Explicit implementation of an ADT
- Cost/Runtime of each operation might vary depending on the implementation and or the computational model (e.g. hash tables implemented on a distr. system perfom very differently)

Anything can be stored...

```
public class Pair⟨A, B⟩ {
protected A a;
protected B b;
public Pair(A a, B b) { this.a = a; this.b = b; }
public void setfirst(A a) { this.a = a; }
public void setsecond(B b) { this.b = b; }
public A first() { return a; }
public B second() { return b; }
public String toString() { return"[" + a + "," + b
+ "]"; }
```

⇒ Can have a data structure storing Pairs.

Abstract Data Structures

Linear data structures:

Data is stored in a sequence

- Array
- Queue: FIFO
- Stack: LIFO

Non-linear data structures:

- Dictionary/Map
- Set Data structures like union find
- Priority Queue (Heap)

Abstract Data Structures

Later:

- **Priority Queue (ADT)**
- Tree-based ADTs with different implementations (with different properties):
  - AVL trees
- Red-black trees
- 2-4 trees

There are many more:

- Graphs as ADT
- Geometric Data Structures

Abstract Data Type: Array

Array

- Sequencially stores a set of elements (of the same type, e.g., int)

Operations:

- Create(n): creates an array A of length n
- A.get(i): returns the element at the i-th position
- A.set(x,i): sets the value of the i-th position to x
- A.size(): returns the size of the array

Some versions may not include this operation

Data Structure Array (Implementation)

Implementation:

Stored in Memory in Consecuitive block

Physical Memory Address=BaseAddress+(Index*ElementSize)

Advantages:

- Fast access to any element/position

Downside:

- Fixed size!!

Confusing: Arrays are ADT and Data Structure at the same time

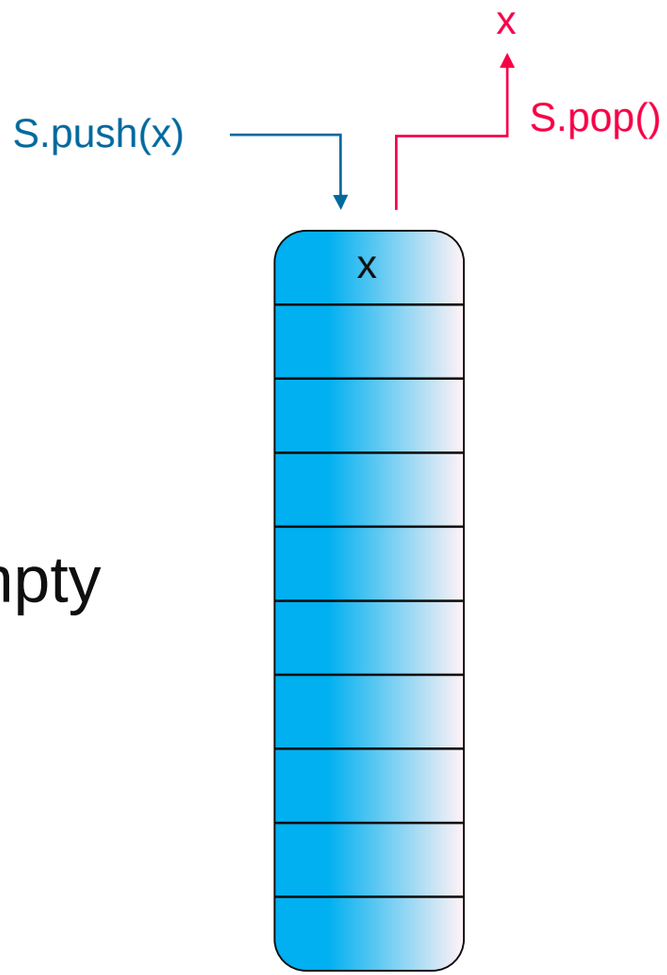
Abstract Data Type: Stack

Stack

- Stores a set/sequence of elements

Operations:

- Create: creates an empty Stack
- S.push(x): appends the element x to the stack
- S.pop(): returns the „top most“ element and deletes it
- S.isEmpty: returns bool, depending wheither the stack is empty



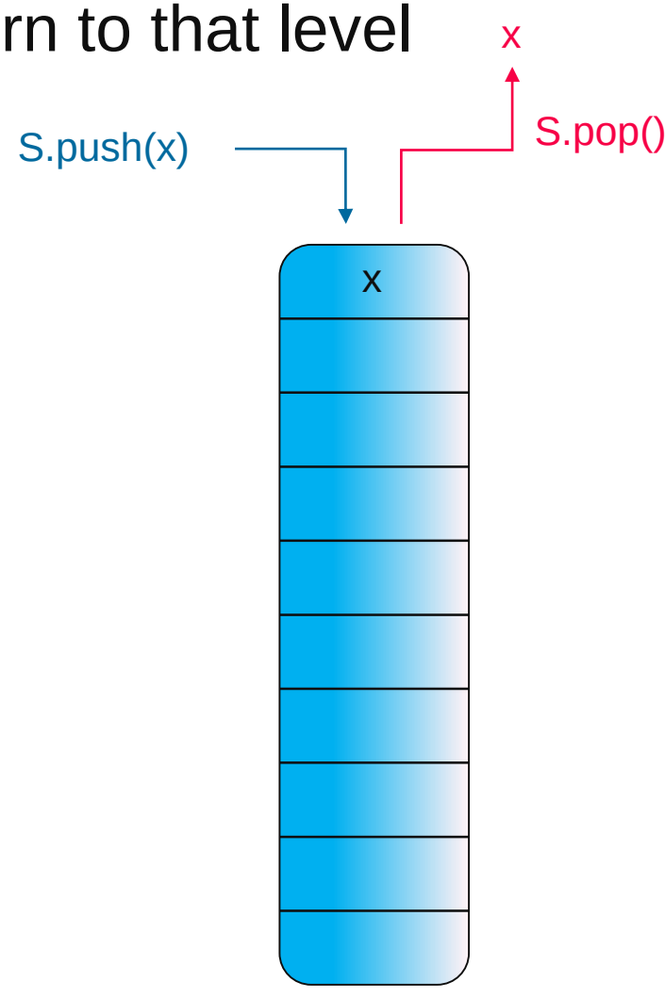
LIFO (= last in first out)

only access to the “top” element

Application Stack

Central data structure for recursion:

- Push the current state onto the stack, load once you return to that level



Graph/Tree traversal:

- Depth First Search (DFS) for graphs

Many more …

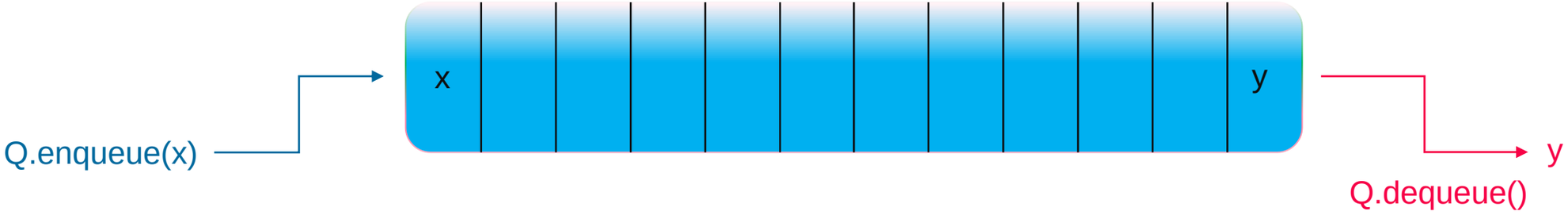
Abstract Data Type: Queue

Queue

- Stores a set/sequence of elements

Operations:

- Create: creates an empty queue
- Q.enqueue(x): appends the element x to the queue
- Q.dequeue(): returns first element of the queue and deletes it
- Q.isEmpty(): returns bool, depending wheiter the queue is empty

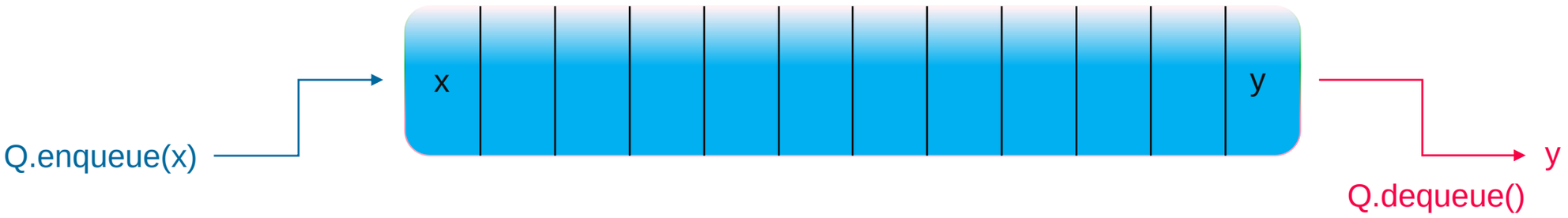


FIFO (= first in first out)

Applications: Queue

Applications:

- First come first serve (scheduling).
- Tree/Graph traversal: Breadth First Search (BFS) for graphs.
- Many more



Implementation: Stack

Array based implementation of Stack:

Operations: create, push, pop, isEmpty

Assumption: Stack will not get longer than N_{MAX}

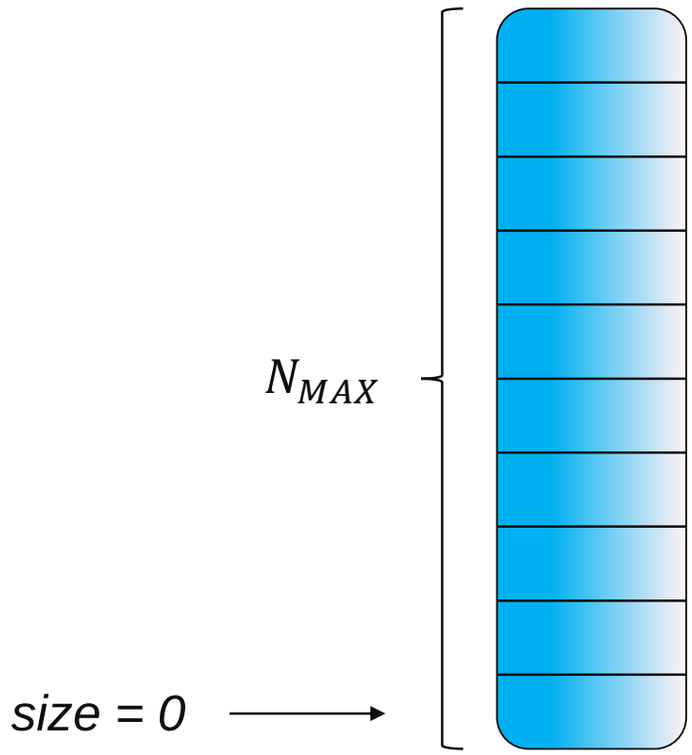
Variables to keep track of the stack:

- *stack*: array of length N_{MAX}
- *size*: number of elements in the stack

create():

stack = new array of length N_{MAX}

size = 0



Implementation: Stack

- S.isEmpty():

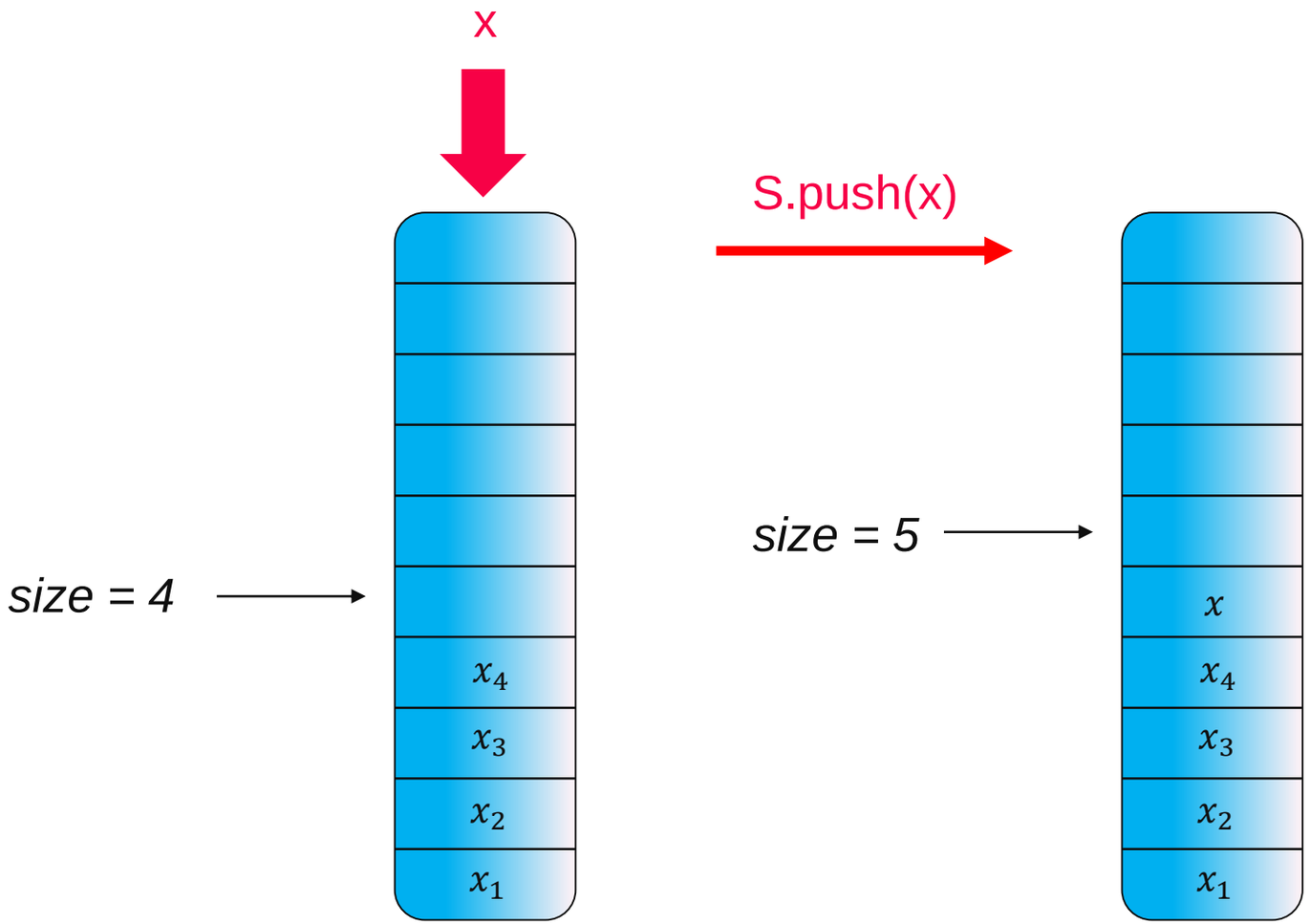
  return (*size* == 0);

- S.push(x):

  if(*size* < N_{MAX}):

  stack[*size*] = x;

  size += 1;



Implementation: Stack

- S.pop():

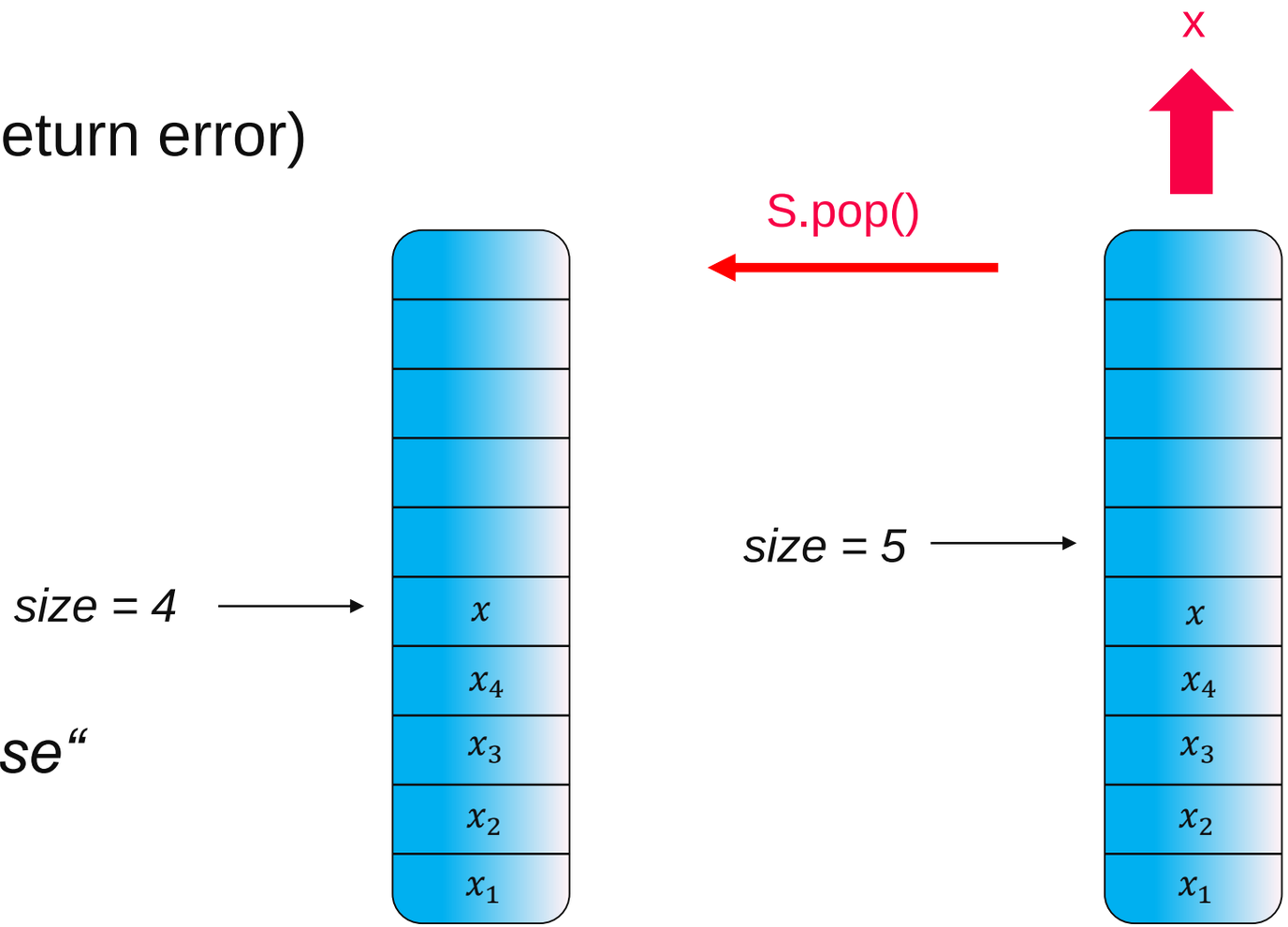
  if(*size* == 0):

  return default value; (or return error)

  else:

  size = *size* - 1;

  return *stack*[*size*];



This implementation does not „erase“ the content at the top position

Analysis: Array based Stack

Runtime of the operations:

- create: $O(1)$
  - If we can allocate memory in $O(1)$, realistic assumption? It depends...
- push: $O(1)$
- pop: $O(1)$
- isEmpty: $O(1)$

Downsides:

- Space complexitiy: $\Theta(N_{MAX})$ independent of size of stack
- The queue can only have N_{MAX} elements

Can we solve those issues?

Lists (ADT)

Want to store a sequence of elements (linear data structure).

Want to be able to have **dynamic** size, so we need a delete operation.

List (ADT):

- create(): creates an empty list
- insert(x,i): inserts the element x at position i to the list
- delete(i): deletes element at i-th position from the list (also possible by value)
- size(): returns the number of elements of the list
- isEmpty(): returns bool, depending wheither the list is empty

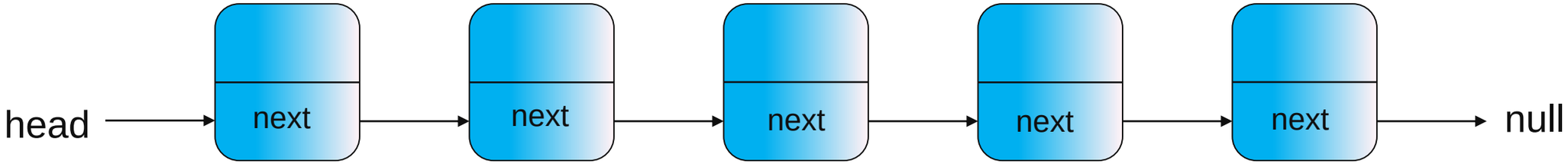
Different to array!

Sometimes it is helpful to have: delete(node x)

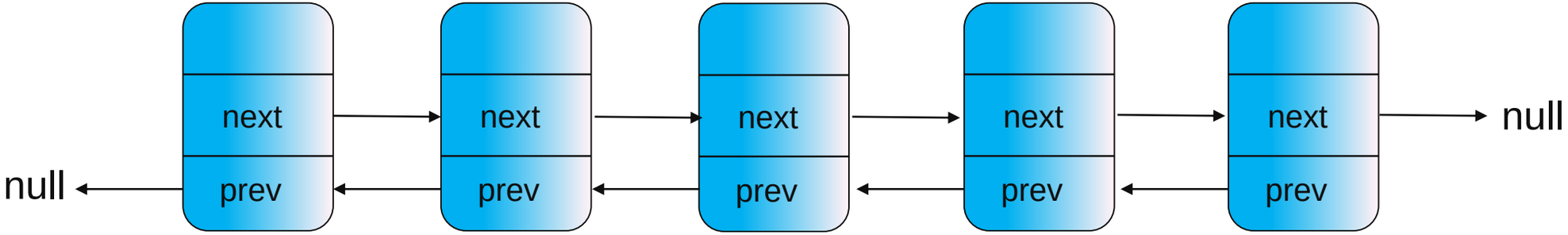
Data Structure (Implementation): Linked Lists

Linear Data Structure

Single Linked List:



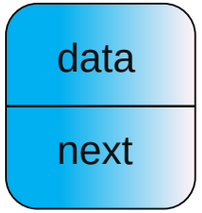
Double Linked List:



Single Linked List: Implementation

- Class for the listelements, also called nodes

- Each node has variables:
  - datatype *data*
  - pointer *next*



data stores the data of the node and *next* points to the next node in the list

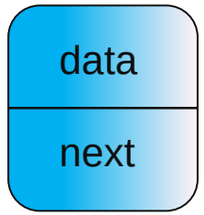
Pyhton:

```
class Node:
    def _init_(self, data=None, next=None):
        self.data = data
        self.next = next
```

Single Linked List: Implementation

Variables to keep track of the Single Linked List:

- pointer *head*
- pointer *tail* ← Dependent on usecase
- int *size*

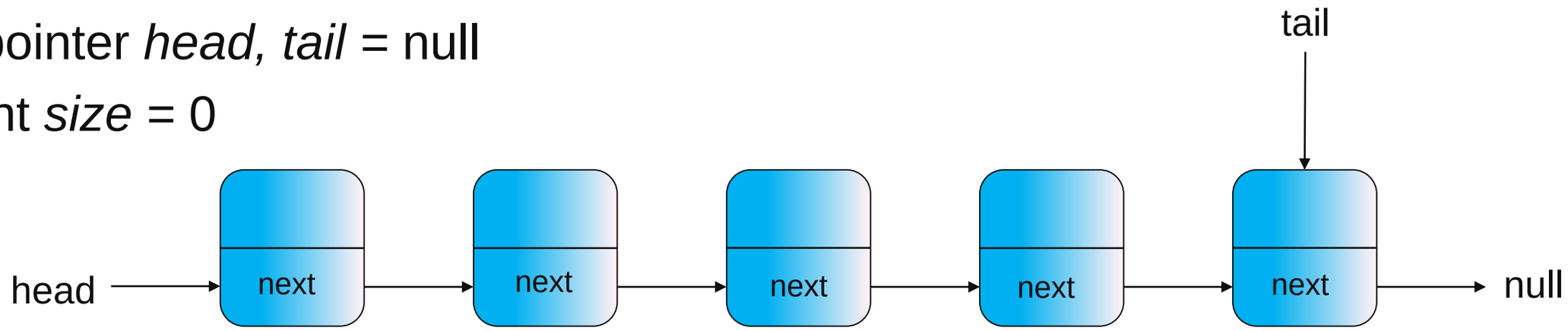


Operations:

- create():

  pointer *head, tail* = null

  int *size* = 0

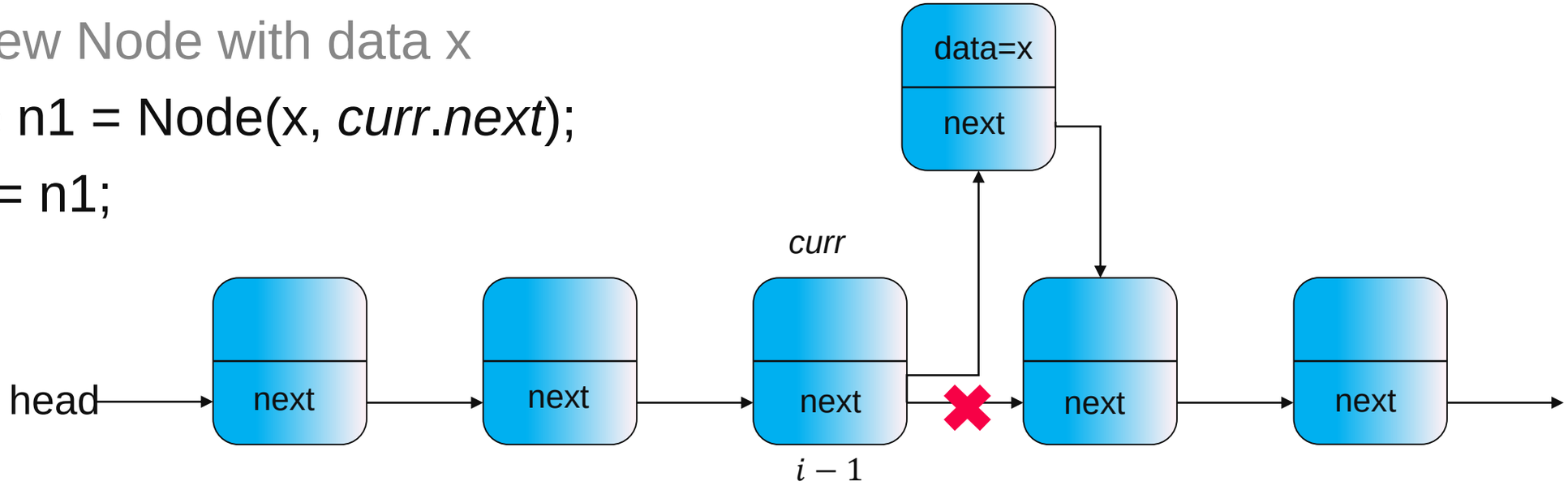


Single Linked List: Implementation

- L.insert(x,i):

```
//traverse through list to position (i-1)
node curr =  *head;
for(int j=1;j<i;j++)
        curr = *curr.next;
//create new Node with data x
new node n1 = Node(x, curr.next);
curr.next = n1;
size++;
```

//Assumes i ≤ length of list
//changes position of later elements



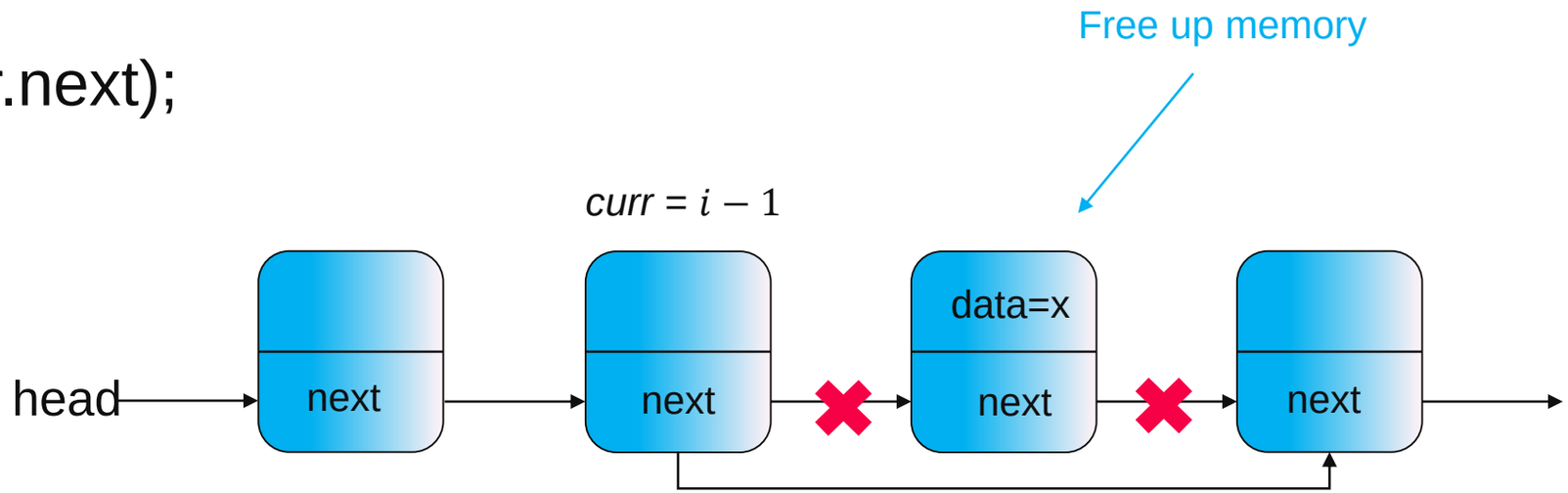
Be careful with edge cases → exercise session

Single Linked List: Implementation

- L.delete(i):

```
//traverse through list until we find node we want to delete
node curr =  *head;
for(int j=1;j<i;j++)
        curr = *curr.next;
//set new pointers
datatype tmp = data.(*curr.next);
curr.next = curr.next.next;
size--;
return tmp;
```

//Assumes i ≤ length of list
//changes position of later elements



Be careful with edge cases → exercise session

Single Linked List: Implementation

- L.size():

  return *size;*

- L.isEmpty():

  return (*head* == null);

Runtimes: With more assumptions you get faster runtimes.

- create(): $O(1)$
- insert(x,i): $O(n)$ $O(1)$, if predecessor is known
- delete(i): $O(n)$ $O(1)$, if predecessor is known
- size(): $O(1)$
- isEmpty(): $O(1)$

Single Linked List: Stack

Single Linked List based implementation of Stack:

Operations: create, push, pop, isEmpty

We just need one Single Linked List (SLL)

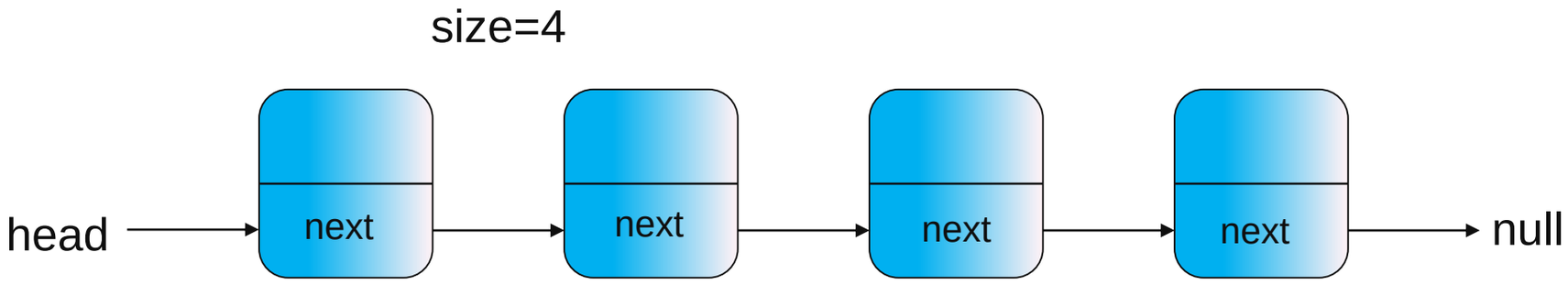
Only with *head* pointer (sometimes called *top*)

create():

stack = new SLL;

pointer *head*

int *size*



Single Linked List: Stack

- S.isEmpty():

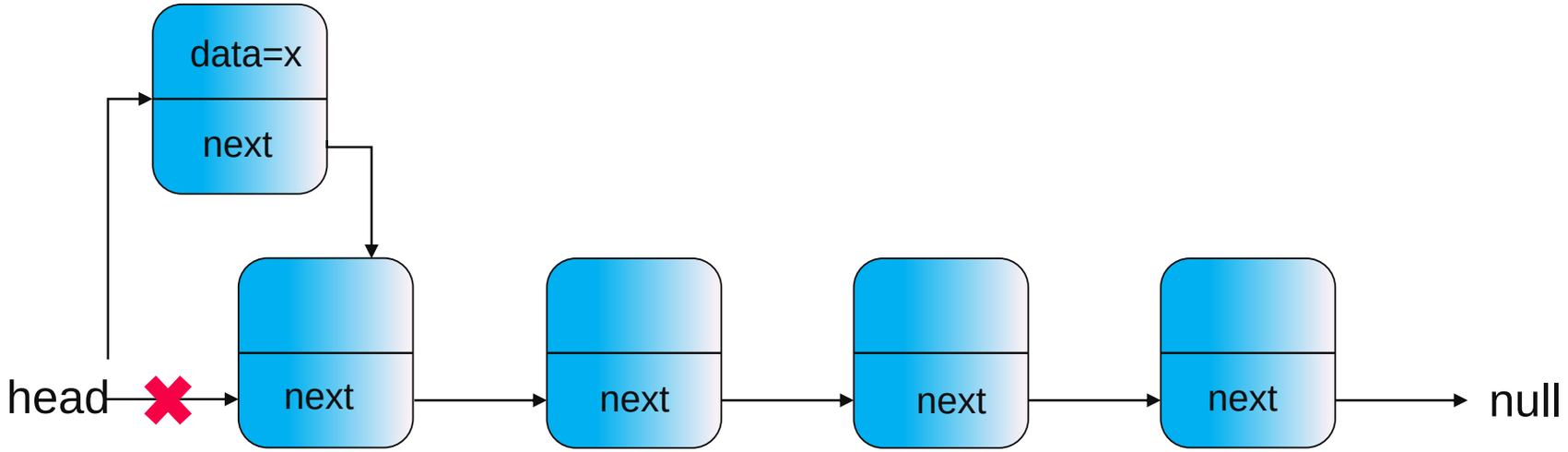
  return (*stack.size*==0);

- S.push(x):

  new node n1 = Node(x,**stack.head*);

  stack.head = n1;

  stack.size++;



Single Linked List: Stack

- S.pop():

  if(*stack.size* == 0):

  return default value; (or return error)

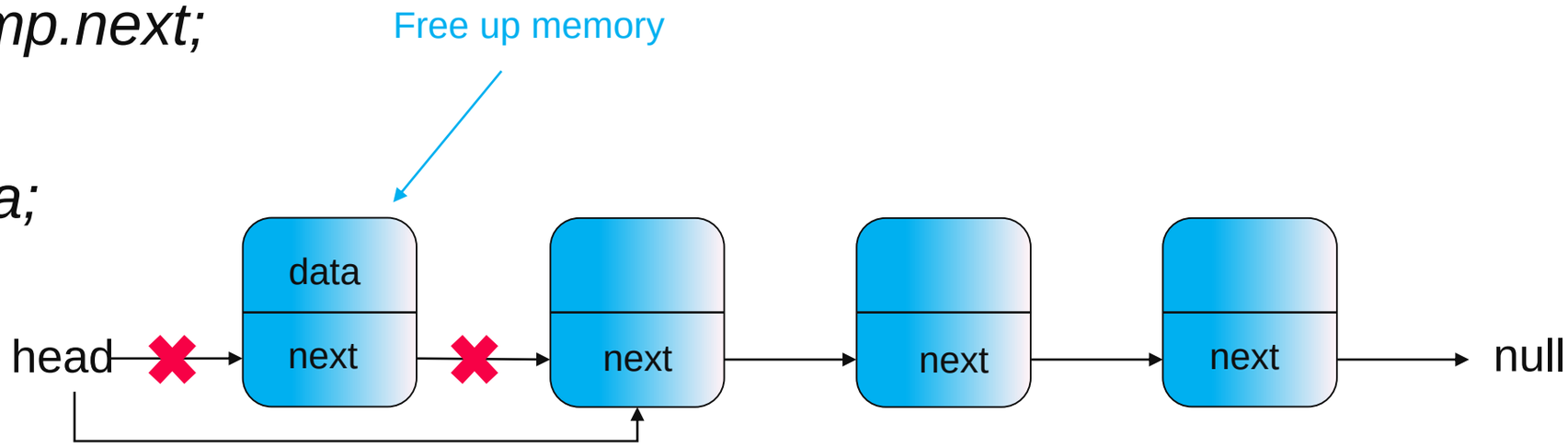
  else:

  node *tmp* = **head;*

  stack.head = tmp.next;

  stack.size--;

  return *tmp.data;*



Single Linked List: Queue

Single Linked List based implementation of Queue:

Operations: create, enqueue, dequeue, isEmpty

We just need one Single Linked List (SLL)

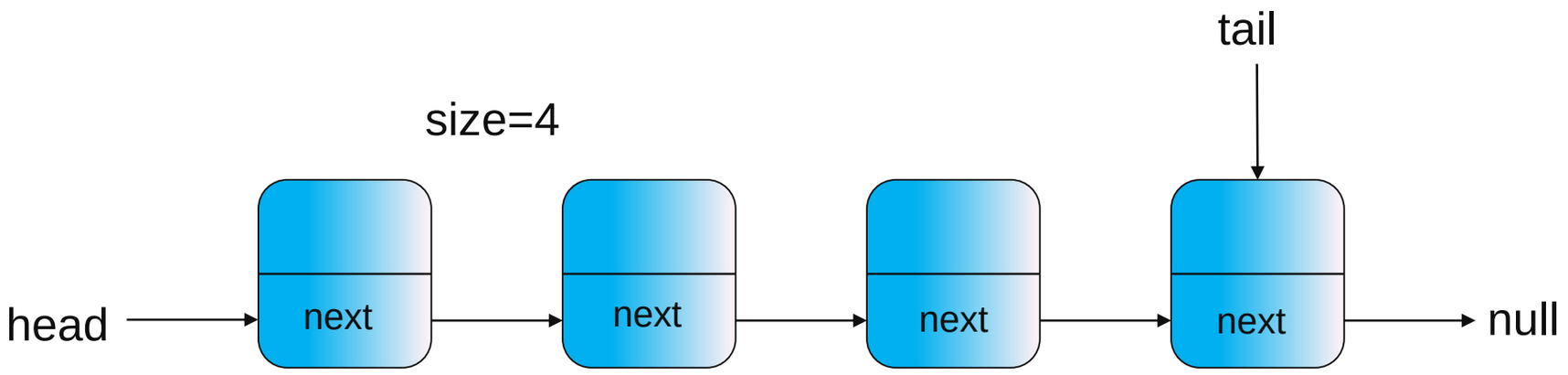
With *head* and *tail* pointer

create():

queue = new SLL;

pointer *head, tail*

int *size*



Single Linked List: Queue

- Q.isEmpty():

  return (*queue.size* == 0);

- Q.enqueue(x):

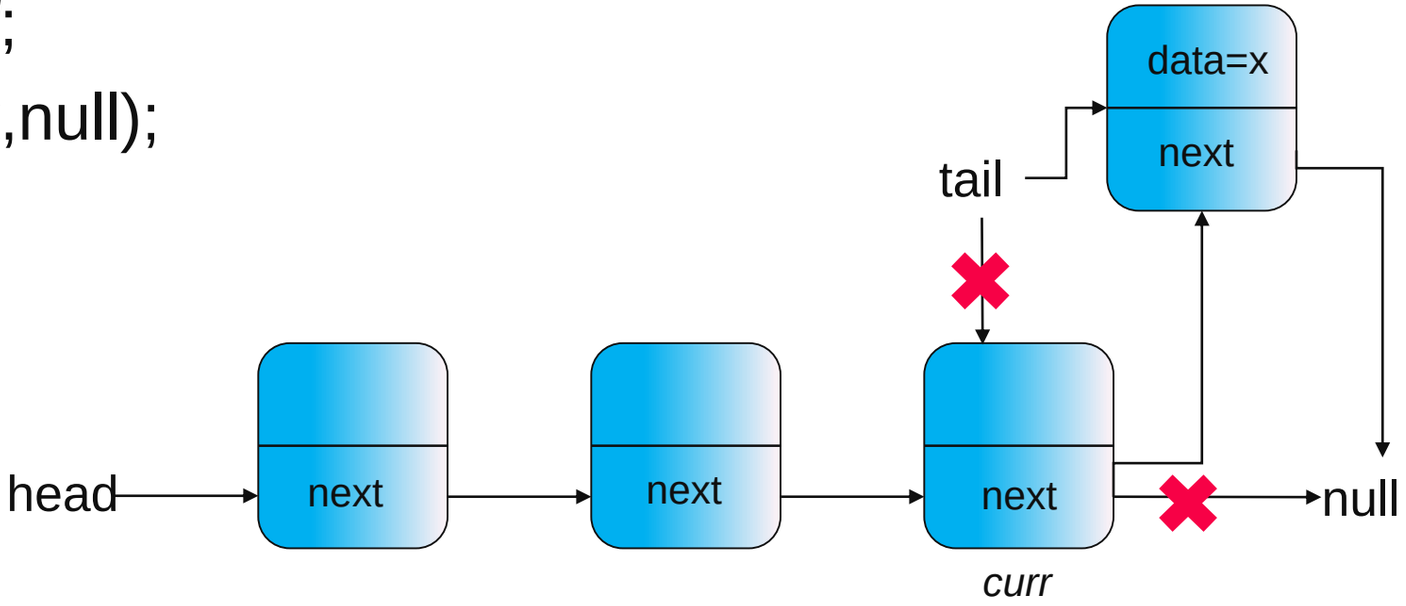
  node *curr* = **queue.tail*;

  new node n1 = Node(x,null);

  curr.next = n1;

  queue.tail = n1;

  queue.size++;



Single Linked List: Queue

- S.dequeue():

  if(*queue.size* == 0):

  return default value; (or return error)

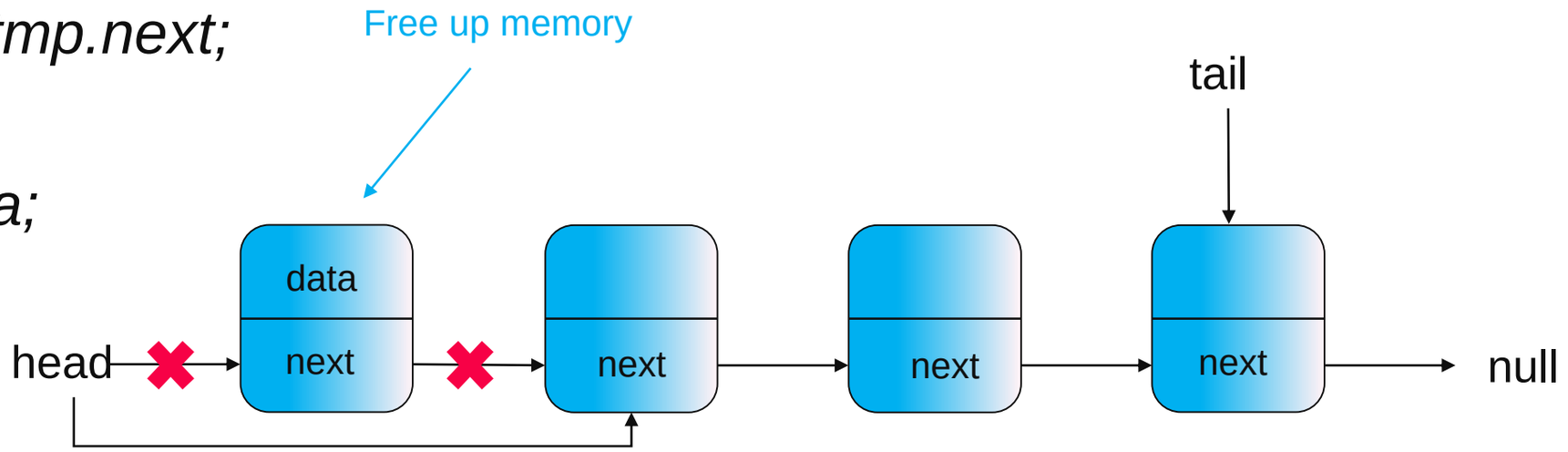
  else:

  node *tmp* = **queue.head;*

  queue.head = tmp.next;

  queue.size--;

  return *tmp.data;*



Analysis: Linked list based implementations

Runtime of the operations (Stack):

- create: $O(1)$
- push: $O(1)$
- pop: $O(1)$
- isEmpty: $O(1)$

Runtime of the operations (Queue):

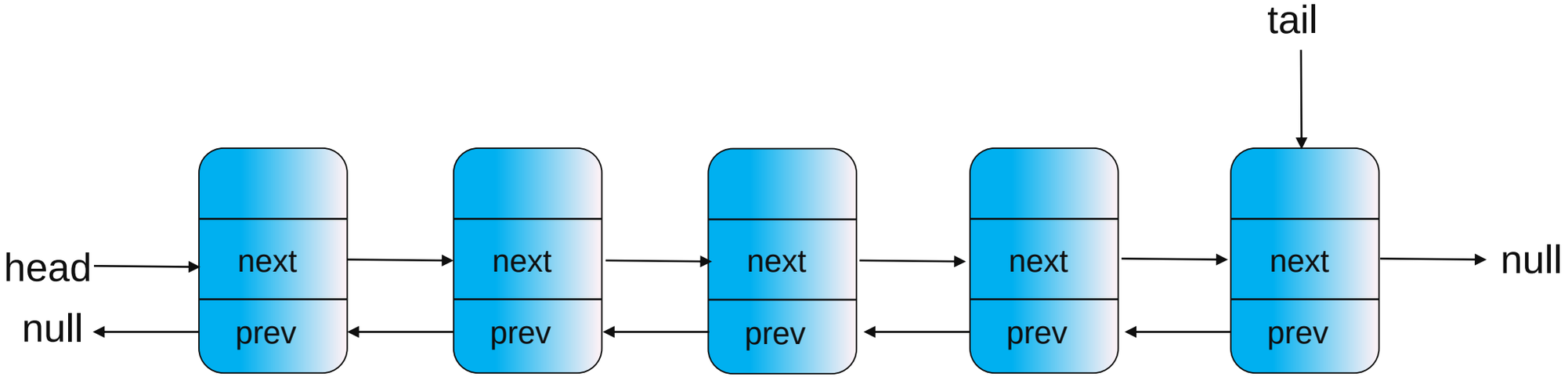
- create: $O(1)$
- enqueue: $O(1)$
- dequeue: $O(1)$
- isEmpty: $O(1)$

... and we have dynamic size!

Doubly Linked List

Why can a double linked list be useful?

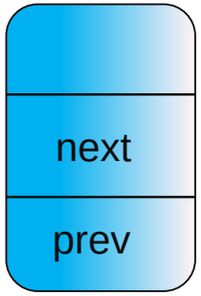
- Can traverse the list in both directions
- Implementation of undo and redo functionality (each node is a state)
  - E.g. browser history management



Implementation: Doubly Linked List

- Class for the listelements, also called nodes

- Each node has variables:
  - datatype *data*
  - pointer *next*
  - pointer *prev*



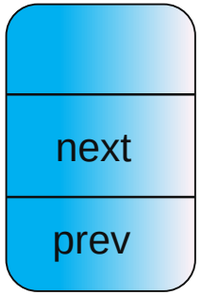
data stores the data of the node and *next* points to the next node in the list and *prev* to the previous node in the list

Implementation: Doubly Linked List

Operations: create, insert(i,x), delete(i), size, isEmpty. (List ADT)
+ insertAfter(i,x), insertBefore(i,x)

Variables to keep track of the Double Linked List:

- pointer *head*
- pointer *tail*
- int *size*

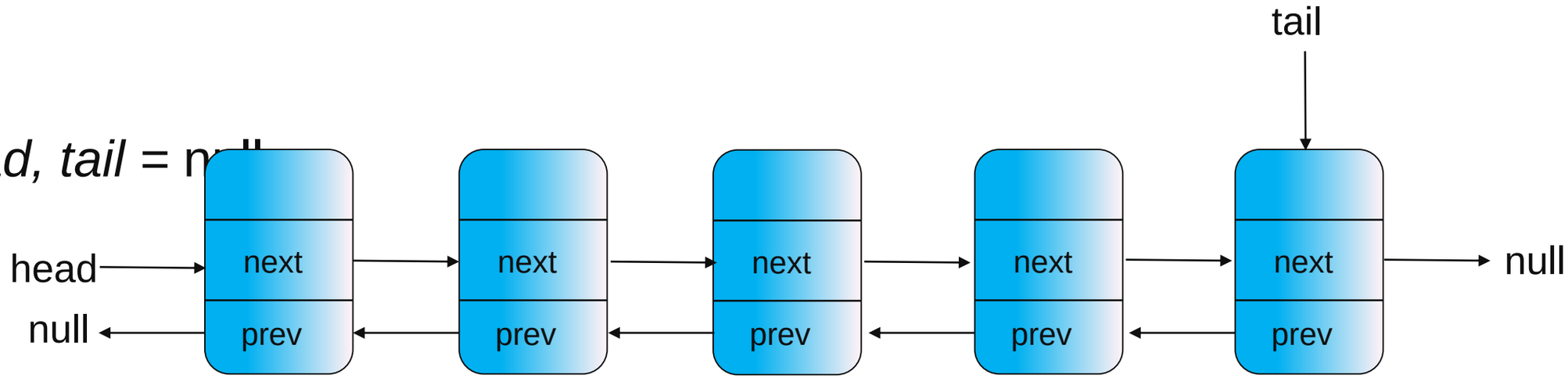


Operations:

- create():

  pointer *head, tail* = null

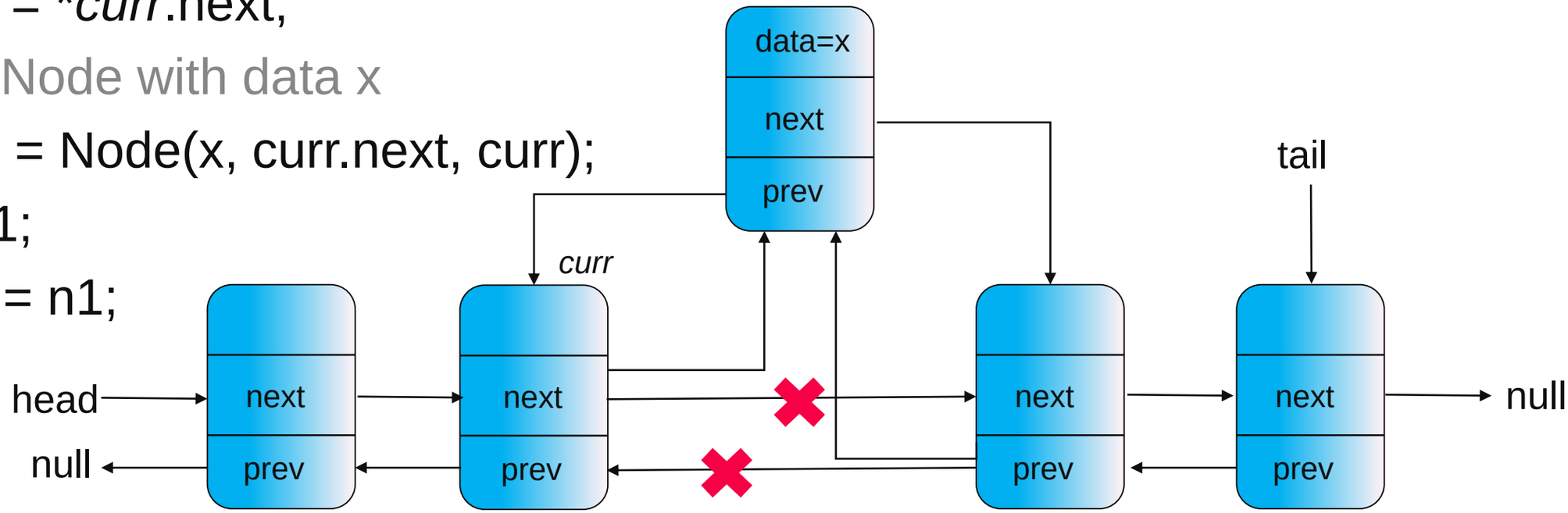
  int *size* = 0



Double Linked List: Implementation

- L.insert(x,i):

```
//traverse through list to position (i-1)
node curr =  *L.head;
for(int j=1;j<i;j++)
        curr = *curr.next;
//create new Node with data x
new node n1 = Node(x, curr.next, curr);
curr.next = n1;
n1.next.prev = n1;
L.size++;
```

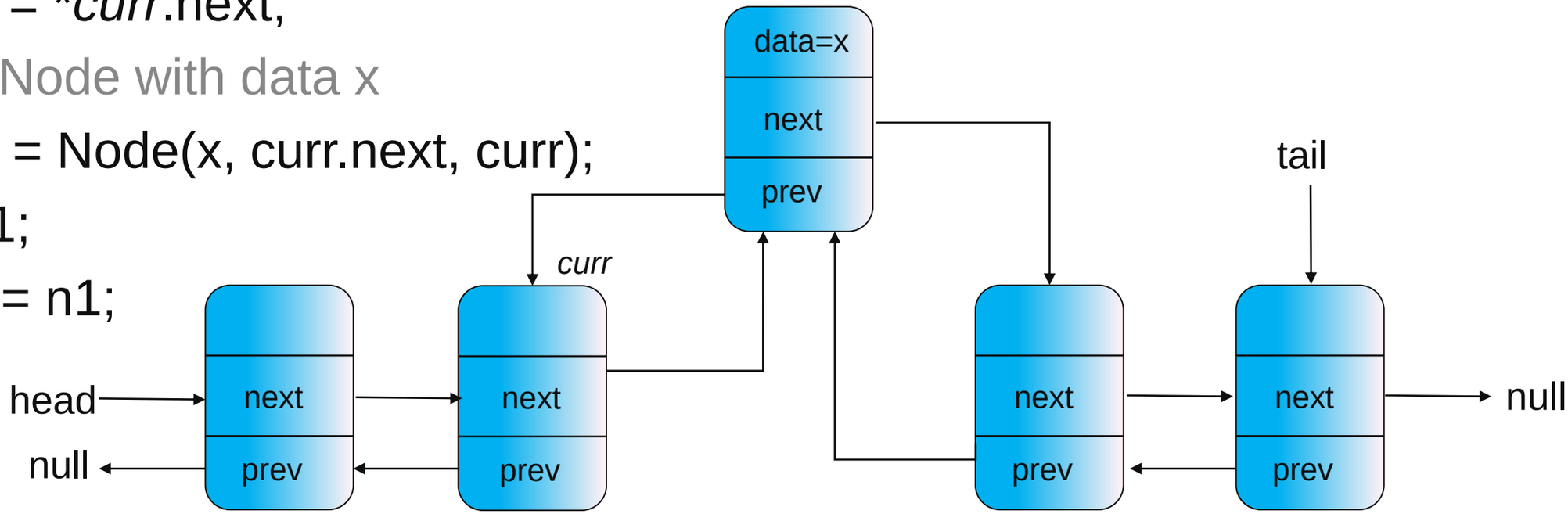


Double Linked List: Implementation

- L.insert(x,i):

```
//traverse through list to position (i-1)
node curr =  *L.head;
for(int j=1;j<i;j++)
        curr = *curr.next;
//create new Node with data x
new node n1 = Node(x, curr.next, curr);
curr.next = n1;
n1.next.prev = n1;
L.size++;
```

//Assumes i ≤ length of list
//changes position of later elements



Be careful with edge cases

Double Linked List: Implementation

- L.delete(i):

  node *curr* = ***L.*head*;

  for(int j=1;j≤i;j++)

  curr = **curr.next*;

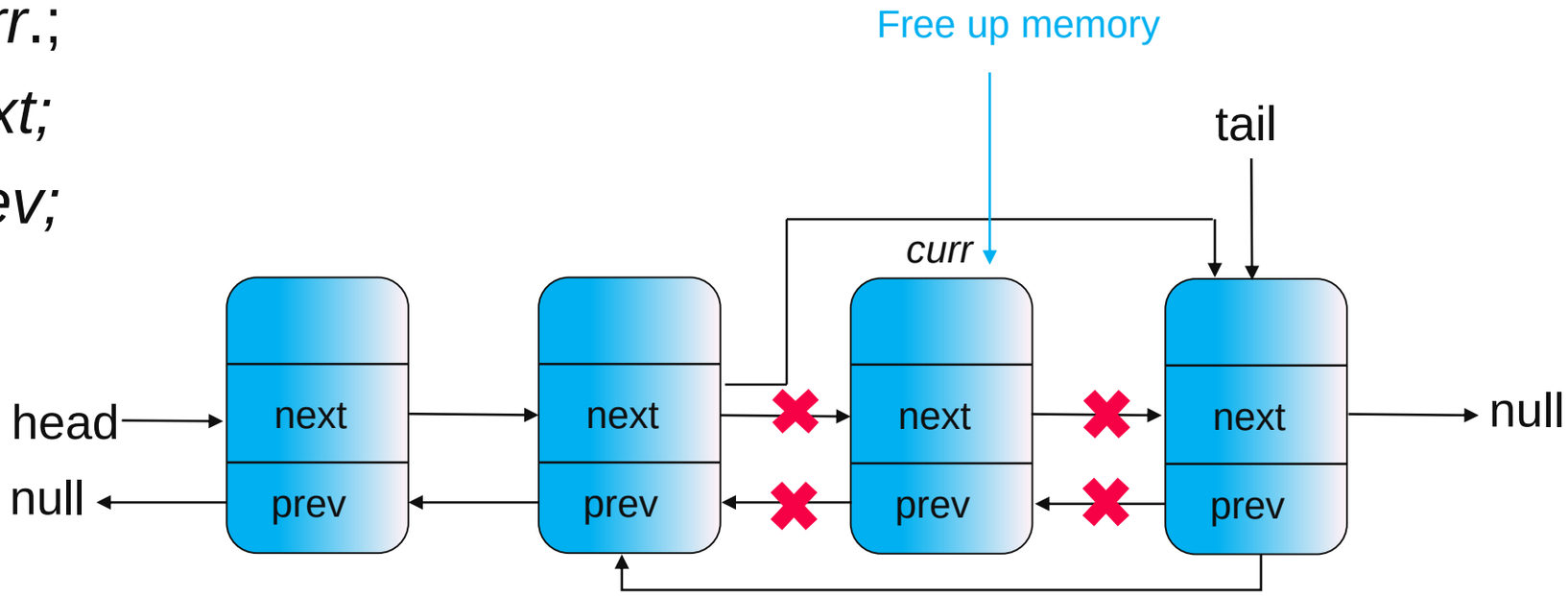
  datatype *tmp* = *data.curr*.;

  curr.prev.next = curr.next;

  curr.next.prev = curr.prev;

  L.*size*--;

  return *tmp;*

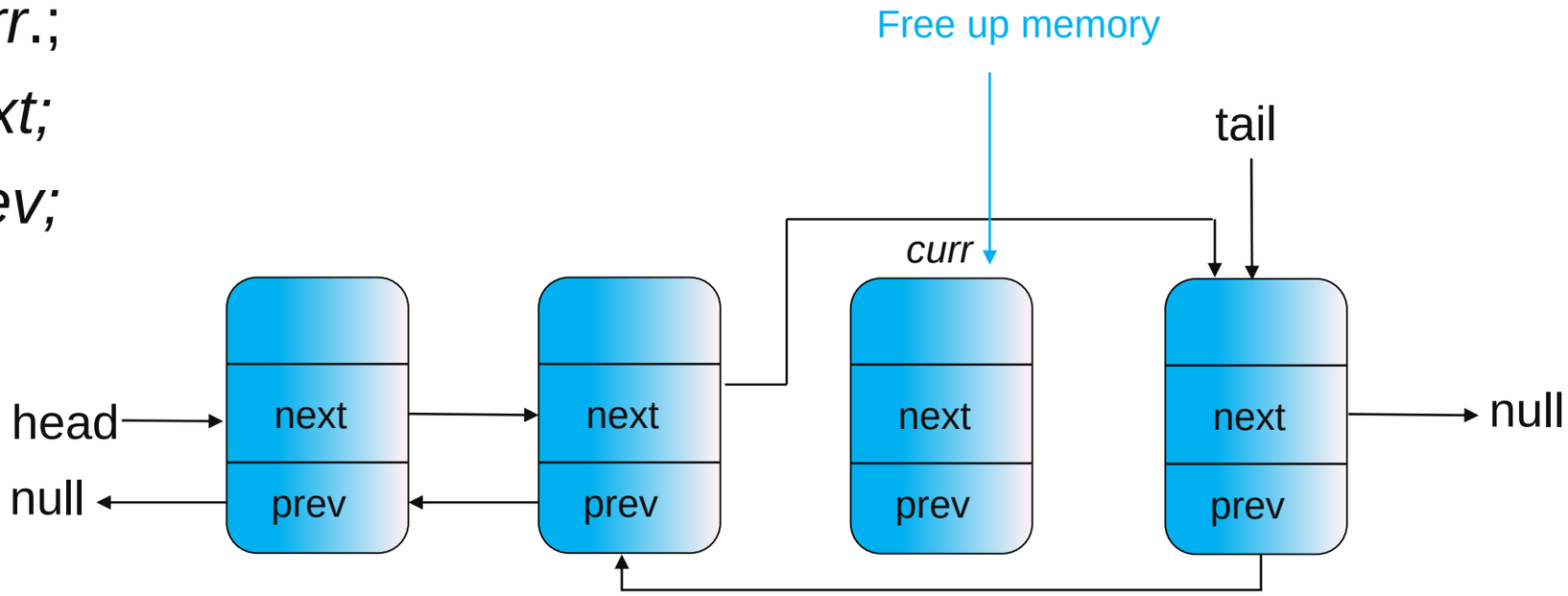


Double Linked List: Implementation

- L.delete(i):

```
node curr =  *L.head;
for(int j=1;j≤i;j++)
        curr = *curr.next;
datatype tmp = data.curr.;
curr.prev.next = curr.next;
curr.next.prev = curr.prev;
L.size--;
return tmp;
```

//Assumes i ≤ length of list
//changes position of later elements



Be careful with edge cases

Double Linked List: Implementation

- size():

  return L.*size*;

- isEmpty():

  return (L.*size*==0);

Runtime of the operations:

- create: $O(1)$
- insert(i,x): $O(n)$
- delete(i): $O(n)$
- isEmpty: $O(1)$
- Size: $O(1)$

Summary

We have seen

- Abstract Data Structure (interface) vs. Data Structure (implementation)

ADTs:

- Array
- Stack
- Queue
- List ADT
- Doubly linked List ADT

Data Structures

- Array
- Single/Doubly linked lists
- Stack via Array
- Stack/Queue via Lists

… many more to come in this course

Important:
Be aware that interface and implementation are not necessarily the same

(Opinions on what's an ADT (interface) vs implementation differ in the literature)